



SOFTWARE REQUIREMENTS SPECIFICATION

Expense Manager System



Document Type	Software Requirements Specification (SRS)
Version	1.0
Status	Draft
Architecture	Multi-Tenant RESTful API
Document Date	March 26, 2026

Multi-Tenant · Secure · Scalable · Clean Architecture

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Expense Manager System — a multi-tenant, web-based platform designed to help individuals and organizations track expenses, manage budgets, and derive actionable financial insights.

The system is intended to:

- Provide users with a centralized platform to record and categorize financial transactions
- Enable budget management with real-time tracking against spending limits
- Deliver analytical insights and reports on spending behavior
- Support multiple users with distinct roles operating within shared workspaces (panels)
- Serve as a scalable foundation for a potential SaaS product

1.2 Scope

The Expense Manager System is a RESTful API backend service that:

- Accepts and processes expense data from web or mobile client applications
- Enforces multi-tenant data isolation through a panel-based architecture
- Applies role-based access control (RBAC) to all operations
- Provides aggregated financial reports and trend analytics
- Is designed for horizontal scalability and production-grade deployment

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
Panel	An isolated workspace (tenant) containing users, expenses, categories, and budgets
User	An authenticated individual with access to one or more panels
Expense	A recorded financial transaction belonging to a panel

Term	Definition
Category	A label used to classify expenses (e.g., Food, Rent, Travel)
Budget	A spending limit applied to a category or the overall panel
Owner	A user with full administrative control over a panel
Editor	A user who can create and modify expenses within a panel
Viewer	A user with read-only access to a panel's data
JWT	JSON Web Token — used for stateless authentication
RBAC	Role-Based Access Control — permission model based on assigned roles
SRS	Software Requirements Specification — this document
API	Application Programming Interface
CRUD	Create, Read, Update, Delete — standard data operations

1.4 Document Conventions

- Requirements are written using the form: "The system shall..."
- Priority levels: MUST (mandatory), SHOULD (recommended), MAY (optional)
- Endpoints are expressed as HTTP METHOD /path
- All field names use snake_case

1.5 Intended Audience

- Backend developers implementing the system
- Frontend/mobile developers consuming the API
- QA engineers designing test cases
- Project managers and stakeholders reviewing scope

2. Overall Description

2.1 Product Perspective

The Expense Manager System functions as the authoritative backend service for all financial data management. It exposes a secure, versioned REST API consumed by frontend clients (web browsers, mobile apps). The system sits within a broader technology stack:

Layer	Technology / Responsibility
Client Layer	Web / Mobile apps consuming the REST API
API Layer	RESTful API — authentication, routing, validation, response formatting
Service Layer	Business logic — expense processing, budget tracking, analytics
Repository Layer	Data access abstraction — queries, transactions, caching
Database Layer	Relational database (PostgreSQL recommended) — persistent storage
Cache Layer	Redis — session tokens, query caching, rate limiting
Notification Layer	Event-driven alerts — budget exceeded, activity summaries

2.2 User Classes and Characteristics

Role	Permissions	Typical User
Owner	Full control: manage panel settings, invite/remove users, assign roles, all CRUD on all data	Business owner, account administrator
Editor	Create, read, update, delete expenses and categories; view reports and budgets	Finance team member, accountant
Viewer	Read-only: view expenses, categories, budgets, and reports	Auditor, manager reviewing spend

2.3 Operating Environment

- Deployed on cloud infrastructure (AWS, GCP, Azure, or equivalent)
- Containerized via Docker; orchestrated via Kubernetes or equivalent
- PostgreSQL 14+ as the primary relational database
- Redis 7+ for caching and session management

- HTTPS required for all client-to-server communication
- API versioning via URL prefix (e.g., /api/v1/)

2.4 Design and Implementation Constraints

- Must follow RESTful principles and return JSON responses
- Authentication via JWT — stateless, no server-side sessions
- All data operations must respect panel-level isolation
- Passwords must be hashed using bcrypt (cost factor ≥ 12)
- Database schema must enforce foreign key constraints
- API responses must include standard error codes and descriptive messages

2.5 Assumptions and Dependencies

- Client applications handle UI rendering; this system provides data only
- Users have reliable internet access
- Email delivery service (SMTP or provider) is available for notifications
- Database backups and infrastructure monitoring are handled at the deployment level
- The system will initially support English locale; i18n is a future enhancement

3. Functional Requirements

Architecture Note: All functional modules operate within a multi-tenant context. Every request is scoped to the authenticated user's panel. Cross-panel data access is strictly forbidden.

3.1 Authentication & Authorization

SECURITY CORE

3.1.1 Features

- User registration with email and password
- Login with credential validation and JWT issuance
- Token refresh mechanism for extended sessions
- Logout with token invalidation
- Role-based access enforcement on all protected endpoints

3.1.2 Requirements

REQ-AUTH-001 (MUST): The system shall validate user credentials against stored bcrypt-hashed passwords during login.

REQ-AUTH-002 (MUST): The system shall issue a signed JWT upon successful authentication containing user_id, panel_id, and role claims.

REQ-AUTH-003 (MUST): The system shall reject requests bearing expired, malformed, or revoked tokens with HTTP 401.

REQ-AUTH-004 (MUST): The system shall enforce role-based access control on all protected endpoints, returning HTTP 403 for unauthorized actions.

REQ-AUTH-005 (MUST): Passwords shall be hashed with bcrypt at a minimum cost factor of 12 before storage.

REQ-AUTH-006 (SHOULD): The system should support access token + refresh token pairs to reduce login frequency.

3.1.3 Validation Rules

Field	Rule
email	Valid email format; unique across the system
password	Minimum 8 characters; must contain at least one uppercase, one lowercase, one digit
username	Alphanumeric + underscores; 3–50 characters; unique across the system

3.2 Panel Management

MULTI-TENANT CORE

3.2.1 Features

- Create a new panel (workspace)
- List all panels a user belongs to
- Update panel name and settings
- Delete a panel (owner only)
- Invite users to a panel by email
- Assign or change user roles within a panel
- Remove users from a panel

3.2.2 Requirements

REQ-PANEL-001 (MUST): A single user account shall be permitted to create and own multiple panels.

REQ-PANEL-002 (MUST): Each panel shall maintain complete data isolation — no expense, category, or budget data shall be accessible across panel boundaries.

REQ-PANEL-003 (MUST): A user may belong to multiple panels simultaneously, each with an independent role assignment.

REQ-PANEL-004 (MUST): Only the panel Owner shall be permitted to invite users, change roles, and delete the panel.

REQ-PANEL-005 (MUST): Panel invitations shall be delivered via email and require acceptance before access is granted.

REQ-PANEL-006 (SHOULD): The system should support a panel activity audit log recording user actions with timestamps.

3.2.3 PanelUser Relationship Model

Attribute	Type	Description
user_id	UUID FK	Reference to the User
panel_id	UUID FK	Reference to the Panel
role	ENUM	'owner' 'editor' 'viewer'
joined_at	TIMESTAMP	When the user accepted the invitation
invited_by	UUID FK	User who sent the invitation

3.3 Expense Management

CORE FEATURE

3.3.1 Features

- Record a new expense with full metadata
- Edit an existing expense (Editor+ role required)
- Delete an expense (Editor+ role required)
- Attach notes and optional file attachments to expenses
- Filter expenses by date range, category, amount range, and creator
- Paginate expense lists for performance

3.3.2 Expense Data Model

Field	Type	Required	Description
id	UUID	Auto	Primary key, auto-generated
panel_id	UUID FK	Yes	Owning panel — enforces tenant isolation
amount	DECIMAL(12,2)	Yes	Positive monetary value
category_id	UUID FK	Yes	Assigned category within the same panel
date	DATE	Yes	Date the expense occurred (not necessarily created)
description	VARCHAR(500)	No	Optional human-readable note

Field	Type	Required	Description
created_by	UUID FK	Auto	User who recorded the expense
created_at	TIMESTAMP	Auto	Server-side creation timestamp
updated_at	TIMESTAMP	Auto	Last modification timestamp

3.3.3 Requirements

REQ-EXP-001 (MUST): The system shall validate that amount is a positive decimal with at most two decimal places.

REQ-EXP-002 (MUST): The system shall validate that category_id references a category belonging to the same panel.

REQ-EXP-003 (MUST): The system shall store created_by automatically from the authenticated user's identity.

REQ-EXP-004 (MUST): The system shall return expenses scoped strictly to the authenticated user's active panel.

REQ-EXP-005 (MUST): Deletion of an expense shall be a soft delete — the record is retained with a deleted_at timestamp for auditability.

REQ-EXP-006 (SHOULD): The system should support pagination with a maximum page size of 100 items.

3.4 Category Management

3.4.1 Features

- System-provided default categories available to all panels
- Create custom categories scoped to a specific panel
- Update or delete custom categories
- Assign a category to an expense

3.4.2 Requirements

REQ-CAT-001 (MUST): Category names shall be unique within a panel (case-insensitive comparison).

REQ-CAT-002 (MUST): Deleting a category shall be prevented if active expenses reference it, unless the caller provides a replacement category.

REQ-CAT-003 (MUST): Default system categories shall be read-only and cannot be modified or deleted by any user.

REQ-CAT-004 (SHOULD): Categories should support an optional color code for client-side display.

3.5 Budget Management

IMPORTANT

3.5.1 Features

- Set a total budget limit for an entire panel
- Set a budget limit for a specific category within a panel
- View current spending vs. budget in real time
- Receive alerts when spending reaches configurable thresholds (e.g., 80%, 100%)

3.5.2 Budget Data Model

Field	Type	Description
id	UUID	Primary key
panel_id	UUID FK	Owning panel
category_id	UUID FK (nullable)	NULL = panel-wide budget; set = category-specific budget
limit_amount	DECIMAL(12,2)	Maximum allowed spending amount
period	ENUM	'monthly' 'quarterly' 'annual' 'custom'
start_date	DATE	Period start (required for 'custom' period)
end_date	DATE	Period end (required for 'custom' period)
alert_threshold	INTEGER	Percentage (0–100) at which an alert is triggered. Default: 80

3.5.3 Requirements

REQ-BUD-001 (MUST): The system shall prevent more than one active budget per panel per period for the same category (or panel-wide).

REQ-BUD-002 (MUST): The system shall calculate current spend against a budget in real time on each relevant API request.

REQ-BUD-003 (MUST): The system shall emit a notification event when cumulative spending in a budget period reaches the configured alert threshold.

REQ-BUD-004 (SHOULD): The system should return a `budget_status` field ("on_track" | "warning" | "exceeded") alongside budget detail responses.

3.6 Reporting & Analytics

HIGH VALUE

3.6.1 Features

- Monthly expense summary (total spend, transaction count, average per transaction)
- Category breakdown with percentage share of total spend
- Spending trends over time (weekly / monthly aggregation)
- Top spending categories for a given period
- Comparison between periods (current month vs. previous month)
- Per-user spending breakdown (Owner/Editor view)

3.6.2 Requirements

REQ-RPT-001 (MUST): All report queries shall be scoped to the authenticated user's panel.

REQ-RPT-002 (MUST): The system shall aggregate expense data efficiently using indexed database queries and materialised views where applicable.

REQ-RPT-003 (MUST): Reports shall accept date range parameters (`from_date`, `to_date`) and default to the current calendar month.

REQ-RPT-004 (SHOULD): Frequently requested reports should be cached in Redis with a configurable TTL (default: 5 minutes).

REQ-RPT-005 (SHOULD): The system should support exporting report data in JSON and CSV formats via query parameter (`?format=csv`).

3.6.3 Sample Report Response Structure

Field	Type	Description
period	Object	{ from_date, to_date, label }
total_amount	Decimal	Sum of all expenses in the period
transaction_count	Integer	Number of expense records
average_transaction	Decimal	total_amount / transaction_count
by_category	Array	[[category_id, name, amount, percentage, count]]
trend	Array	[[date, amount]] — daily or weekly aggregation
vs_previous_period	Object	{ amount, change_percent, direction }
budget_utilization	Array	[[budget_id, limit, spent, remaining, status]]

3.7 Notifications

3.7.1 Notification Types

Event	Trigger Condition	Delivery Method	Recipients
Budget Warning	Spend reaches alert_threshold %	Email + in-app	Owner, Editor
Budget Exceeded	Spend exceeds limit_amount	Email + in-app	Owner, Editor
User Invited	Panel invitation created	Email	Invited user
Role Changed	User role updated in panel	Email + in-app	Affected user
Expense Deleted	Expense soft-deleted by another user	In-app	Creator of expense

3.7.2 Requirements

REQ-NOT-001 (MUST): Budget-related notifications shall be triggered synchronously via an event bus after each expense creation or update.

REQ-NOT-002 (MUST): Email notifications shall include the panel name, the triggering event, and a direct link to the relevant section of the application.

REQ-NOT-003 (SHOULD): Users should be able to configure their notification preferences (opt-in/out per notification type) from their profile settings.

4. Non-Functional Requirements

4.1 Performance

Metric	Target	Notes
API response time (p95)	< 200ms	Excluding report/analytics endpoints
Report generation time	< 1 second	With Redis caching; < 3s cold
Throughput	≥ 500 req/s	Under normal load, single instance
Database query time	< 50ms (p95)	For indexed, scoped queries
Uptime SLA	99.9%	~8.7 hours downtime/year maximum

4.2 Security

- All passwords hashed with bcrypt, cost factor ≥ 12
- JWT tokens signed with RS256 (asymmetric) algorithm; short expiry (15 minutes) + refresh token
- All API inputs sanitized and validated before processing
- SQL injection prevented via parameterized queries / ORM
- Rate limiting applied per IP and per authenticated user
- HTTPS enforced for all endpoints; HSTS headers set
- Sensitive fields (passwords) never included in API responses
- Panel isolation enforced at the service layer, not only at the query layer

4.3 Scalability

- Stateless API design enabling horizontal scaling behind a load balancer
- Database connection pooling (PgBouncer or equivalent)
- Redis cluster for distributed caching and session management
- Background job queue (e.g., BullMQ, Celery) for notifications and report generation
- Database read replicas for analytics and reporting queries

4.4 Reliability & Data Integrity

- All multi-step operations wrapped in database transactions

- Soft deletes for all financial records — data is never permanently destroyed
- Foreign key constraints enforced at the database level
- Automated database backups with point-in-time recovery
- Idempotent API design for create/update endpoints where feasible

4.5 Maintainability

- Layered architecture: Controller → Service → Repository → Database
- SOLID principles applied throughout the codebase
- All public interfaces and business logic covered by unit tests ($\geq 80\%$ coverage)
- Integration tests for all API endpoints
- Structured logging (JSON) with correlation IDs for request tracing
- OpenAPI / Swagger documentation auto-generated from code annotations

5. Data Model

5.1 Entity Overview

Entity	Primary Key	Key Relationships	Description
User	uuid id	Has many PanelUser	Registered account with credentials
Panel	uuid id	owner_id → User; has many PanelUser, Expense, Category, Budget	Isolated tenant workspace
PanelUser	(user_id, panel_id)	FK to User, FK to Panel	Junction table with role assignment
Expense	uuid id	panel_id → Panel; category_id → Category; created_by → User	Financial transaction record
Category	uuid id	panel_id → Panel; has many Expense, Budget	Expense classification label
Budget	uuid id	panel_id → Panel; category_id → Category (nullable)	Spending limit for a period
Notification	uuid id	user_id → User; panel_id → Panel	System-generated alert or activity item

5.2 Key Constraints

- (panel_id, category_name) — UNIQUE: category names must be unique per panel
- (panel_id, category_id, period, start_date) — UNIQUE: prevents duplicate budgets
- (user_id, panel_id) — UNIQUE on PanelUser: a user has exactly one role per panel
- expense.amount — CHECK: amount > 0
- budget.limit_amount — CHECK: limit_amount > 0
- budget.alert_threshold — CHECK: alert_threshold BETWEEN 0 AND 100

5.3 Indexing Strategy

Table	Index	Reason
expense	idx_expense_panel_date	Scoped date-range queries for reports
expense	idx_expense_panel_category	Category breakdown analytics

Table	Index	Reason
expense	idx_expense_created_by	Per-user spending queries
paneluser	idx_paneluser_user	Fast panel listing per user
budget	idx_budget_panel_category	Budget lookup on expense insert
category	idx_category_panel	Category validation on expense insert

6. API Endpoints

6.1 Authentication

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	None	Register a new user account
POST	/api/v1/auth/login	None	Authenticate and receive JWT pair
POST	/api/v1/auth/refresh	Refresh token	Exchange refresh token for new access token
POST	/api/v1/auth/logout	Bearer JWT	Invalidate current token
GET	/api/v1/auth/me	Bearer JWT	Get authenticated user profile

6.2 Panels

Method	Endpoint	Role	Description
POST	/api/v1/panels	Any user	Create a new panel (caller becomes owner)
GET	/api/v1/panels	Any user	List all panels the user belongs to
GET	/api/v1/panels/:id	Member	Get panel details and member list
PUT	/api/v1/panels/:id	Owner	Update panel name/settings
DELETE	/api/v1/panels/:id	Owner	Delete panel and all associated data
POST	/api/v1/panels/:id/invite	Owner	Invite a user by email with role
PUT	/api/v1/panels/:id/members/:userId/role	Owner	Change a member's role
DELETE	/api/v1/panels/:id/members/:userId	Owner	Remove a member from panel

6.3 Expenses

Method	Endpoint	Role	Description
POST	/api/v1/expenses	Editor+	Record a new expense
GET	/api/v1/expenses	Viewer+	List expenses with filters and pagination
GET	/api/v1/expenses/:id	Viewer+	Get a single expense by ID
PUT	/api/v1/expenses/:id	Editor+	Update an existing expense
DELETE	/api/v1/expenses/:id	Editor+	Soft-delete an expense

6.4 Categories

Method	Endpoint	Role	Description
GET	/api/v1/categories	Viewer+	List categories available in panel
POST	/api/v1/categories	Editor+	Create a custom category
PUT	/api/v1/categories/:id	Editor+	Update a custom category
DELETE	/api/v1/categories/:id	Editor+	Delete a custom category

6.5 Budgets

Method	Endpoint	Role	Description
GET	/api/v1/budgets	Viewer+	List all budgets for the active panel
POST	/api/v1/budgets	Owner	Create a new budget
PUT	/api/v1/budgets/:id	Owner	Update budget limit or threshold
DELETE	/api/v1/budgets/:id	Owner	Remove a budget
GET	/api/v1/budgets/:id/status	Viewer+	Get real-time spend vs. budget

6.6 Reports

Method	Endpoint	Role	Description
GET	/api/v1/reports/summary	Viewer+	Overall expense summary for a period
GET	/api/v1/reports/monthly	Viewer+	Month-by-month breakdown
GET	/api/v1/reports/categories	Viewer+	Spending breakdown by category
GET	/api/v1/reports/trends	Viewer+	Time-series trend data (daily/weekly)
GET	/api/v1/reports/comparison	Viewer+	Current vs. previous period comparison
GET	/api/v1/reports/export	Viewer+	Download report as CSV or JSON

6.7 Standard Response Format

Convention: All API responses follow a consistent envelope structure. Success responses use 2xx status codes; error responses include a code and human-readable message.

Scenario	HTTP Status	Response Body
Success (data)	200 OK	{ "success": true, "data": { ... } }
Created	201 Created	{ "success": true, "data": { ... } }
No content	204 No Content	(empty body)
Validation error	422 Unprocessable Entity	{ "success": false, "code": "VALIDATION_ERROR", "errors": [...] }
Authentication failed	401 Unauthorized	{ "success": false, "code": "UNAUTHORIZED" }
Permission denied	403 Forbidden	{ "success": false, "code": "FORBIDDEN" }
Resource not found	404 Not Found	{ "success": false, "code": "NOT_FOUND" }
Server error	500 Internal Server Error	{ "success": false, "code": "INTERNAL_ERROR" }

7. Future Enhancements

7.1 Planned Features

Feature	Priority	Description
Recurring Expenses	High	Auto-generate expenses on a defined schedule (daily, weekly, monthly)
PDF / Excel Export	High	Export full reports and expense lists to PDF or XLSX format
Mobile App Support	High	Optimized API responses and push notification support for mobile clients
AI Spending Insights	Medium	ML-based anomaly detection and personalized savings recommendations
Bank API Integration	Medium	Import transactions directly from banking providers via Open Banking APIs
Multi-currency Support	Medium	Store expenses in original currency with real-time conversion to base currency
Receipt Scanning	Medium	OCR-based receipt parsing to auto-populate expense fields
Approval Workflows	Low	Expense submission and approval flow for organizational use cases
Audit Log	Low	Full immutable audit trail of all data changes per panel
Webhooks	Low	Outbound webhooks for budget events and new expenses
i18n / Localization	Low	Multi-language and locale-specific number/date formatting

7.2 Technical Roadmap

- v1.0 — Core authentication, panels, expenses, categories, basic budgets
- v1.1 — Reporting & analytics, Redis caching, CSV export
- v1.2 — Recurring expenses, PDF export, notification preferences
- v2.0 — AI insights, bank API integration, multi-currency support
- v2.1 — Mobile push notifications, receipt scanning, approval workflows

8. Appendix

8.1 Technology Stack Recommendation

Layer	Recommended Technology	Rationale
Runtime	Python Django, (FastAPI)	Strong async support, rich ecosystem, type safety
Framework	Django/ FastAPI	Mature, performant, excellent middleware support
Database	PostgreSQL 14+	ACID compliance, JSON support, mature tooling
ORM	Prisma / TypeORM (TS) or SQLAlchemy (Python)	Type-safe queries, schema migrations
Cache	Redis 7+	In-memory speed, pub/sub for events, cluster support
Auth	JWT (RS256) + bcrypt	Industry standard; stateless; asymmetric signing
Queue	Celery (Python)	Reliable background jobs for notifications
Testing	Pytest (Python)	Unit + integration; mocking; coverage
Docs	Swagger / OpenAPI 3.0	Auto-generated; interactive; client SDK generation
Containerization	Docker + Docker Compose	Reproducible environments; production-ready

8.2 Revision History

Version	Date	Author	Description
1.0	Mar 26, 2026	—	Initial release of SRS document

Build this right. Build it once.

Multi-tenant · Secure · Scalable · Clean Architecture · Production-Ready